

BC Experiment-11

Title:

Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim:

To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Prerequisites:

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker
- Git
- Node.js & npm

Procedure:

Step 1: Update the System

Command:

```
sudo apt update  
sudo apt upgrade -y
```

```
anshsarfare16@cloudshell:~$ sudo apt update  
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease  
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]  
Hit:3 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease  
Hit:4 http://archive.ubuntu.com/ubuntu noble InRelease  
Get:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]  
Get:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]  
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2,377 kB]  
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2,155 kB]  
Hit:9 https://packages.cloud.google.com/apt gcsfuse-noble InRelease  
Hit:10 https://packages.cloud.google.com/apt cloud-sdk InRelease  
Hit:11 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease  
Get:12 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]  
Hit:13 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease  
Fetched 4,914 kB in 6s (778 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
12 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Explanation:

This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies

Command:

```
sudo apt install jq curl git nodejs npm -y
node -v
npm -v
git --version
docker --version
docker ps
```

```
anshsarfare16@cloudshell:~$ node -v
v18.19.1
anshsarfare16@cloudshell:~$ npm -v
9.2.0
anshsarfare16@cloudshell:~$ git --version
git version 2.43.0
anshsarfare16@cloudshell:~$ docker --version
Docker version 29.4.0, build 9d7ad9f
anshsarfare16@cloudshell:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
```

Explanation:

This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while **docker ps** ensures Docker is running properly.

Step 3: Clone Fabric Samples

Command:

```
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
```

```

anshsarfare16@cloudshell:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (238/238), done.
remote: Compressing objects: 100% (129/129), done.
remote: Total 15618 (delta 161), reused 109 (delta 109), pack-reused 15380 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 17.07 MiB/s, done.
Resolving deltas: 100% (8809/8809), done.
anshsarfare16@cloudshell:~$ cd fabric-samples

```

Explanation:

This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:

```

curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

```

```

anshsarfare16@cloudshell:~/fabric-samples$ curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric

Pull Hyperledger Fabric binaries

==> Downloading version 2.5.15 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
==> Will unpack to: /home/anshsarfare16/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 120M  100 120M    0     0 29.5M      0  0:00:04  0:00:04 --:--:-- 33.3M
==> Done.
==> Downloading version 1.5.17 platform specific fabric-ca-client binary
==> Downloading: https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
==> Will unpack to: /home/anshsarfare16/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 31.6M  100 31.6M    0     0 16.2M      0  0:00:01  0:00:01 --:--:-- 36.4M
==> Done.

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer cenv baseos
==> Pulling fabric Images
==> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
d41d93850539: Pull complete
b1cba2e842ca: Pull complete
10a7c8361220: Pull complete
f8c55d757df2: Pull complete
eb2bee6aa5bd: Pull complete

```

Pull Hyperledger Fabric docker images

```
FABRIC_IMAGES: peer orderer ccenv baseos
==> Pulling fabric Images
==> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
==> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
==> ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
==> ghcr.io/hyperledger/fabric-baseos:2.5.15
2.5.15: Pulling from hyperledger/fabric-baseos
Digest: sha256:654bd4554bf97c70902e56d530294795372518da7ae2a7cdfccbe397a878c513
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
==> Pulling fabric ca Image
==> ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
==> List out hyperledger images
```

Explanation:

This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command:

docker images | grep fabric

```
anshsarfarel6@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15        86dded7eda92      1GB       255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15      9972c209be47      178MB     49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15         3e25adc31a75      230MB     66.2MB
hyperledger/fabric-baseos:2.5                  654bd4554bf9      250MB     80.4MB
hyperledger/fabric-baseos:2.5.15               654bd4554bf9      250MB     80.4MB
hyperledger/fabric-baseos:latest               654bd4554bf9      250MB     80.4MB
hyperledger/fabric-ca:1.5                      453a0a727e82      381MB     123MB
hyperledger/fabric-ca:1.5.17                   453a0a727e82      381MB     123MB
hyperledger/fabric-ca:latest                   453a0a727e82      381MB     123MB
hyperledger/fabric-ccenv:2.5                   86dded7eda92      1GB       255MB
hyperledger/fabric-ccenv:2.5.15                86dded7eda92      1GB       255MB
hyperledger/fabric-ccenv:latest                86dded7eda92      1GB       255MB
hyperledger/fabric-orderer:2.5                 9972c209be47      178MB     49.5MB
hyperledger/fabric-orderer:2.5.15              9972c209be47      178MB     49.5MB
hyperledger/fabric-orderer:latest              9972c209be47      178MB     49.5MB
hyperledger/fabric-peer:2.5                    3e25adc31a75      230MB     66.2MB
hyperledger/fabric-peer:2.5.15                 3e25adc31a75      230MB     66.2MB
hyperledger/fabric-peer:latest                 3e25adc31a75      230MB     66.2MB
```

Explanation:

This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and orderer are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables**Command:**

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
```

```
anshsarfare16@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
anshsarfare16@cloudshell:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
anshsarfare16@cloudshell:~/fabric-samples$ peer version
peer:
Version: v2.5.15
Commit SHA: 83c7930
Go version: go1.26.0
OS/Arch: linux/amd64
Chaincode:
Base Docker Label: org.hyperledger.fabric
Docker Namespace: hyperledger
```

Explanation:

This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files.

Running `peer version` verifies that the setup is successful.

Step 7: Start Network and Create Channel**Command:**

```
cd ~/fabric-samples/test-network
./network.sh up createChannel
```

```

anshsarfare16@cloudshell:~/fabric-samples/test-network$ ./network.sh up createChannel
Using docker and docker-composeic-samples/test-network$ ./network.sh up createChannel
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using d
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/anshsarfare16/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] Running 3/7
  :: Network fabric_test                                Creat...                1.7s
  :: Volume "compose_peer0.org1.example.com"            Created                 1.5s
  :: Volume "compose_peer0.org2.example.com"            Created                 1.5s
  :: Volume "compose_orderer.example.com"               Created                 1.5s
  ✓ Container orderer.example.com                      Started                 1.4s
  ✓ Container peer0.org1.example.com                    Started                 1.5s
  ✓ Container peer0.org2.example.com                    Started                 1.5s

Status: 201
{
  "name": "mychannel",
  "url": "/participation/v1/channels/mychannel",
  "consensusRelation": "consenter",
  "status": "active",
  "height": 1
}

Channel 'mychannel' created
Joining org1 peer to the channel...
Using organization 1
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-09 17:35:34.429 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:34.463 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted pro
Joining org2 peer to the channel...
Using organization 2
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-09 17:35:37.555 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:37.592 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted pro
Setting anchor peer for org1...
Using organization 1
Fetching channel config for channel mychannel
Using organization 1
Fetching the most recent configuration block for the channel
++ peer channel fetch config /home/anshsarfare16/fabric-samples/test-network/channel-artifac
tls --cafile /home/anshsarfare16/fabric-samples/test-network/organizations/ordererOrganizati
2026-04-09 17:35:37.685 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:37.690 UTC 0002 INFO [cli.common] readBlock -> Received block: 0
2026-04-09 17:35:37.691 UTC 0003 INFO [channelCmd] fetch -> Retrieving last config block: 0
2026-04-09 17:35:37.693 UTC 0004 INFO [cli.common] readBlock -> Received block: 0

```

Explanation:

This step initializes the blockchain network by starting Docker containers for peers and orderers.

It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode

Command:

```
./network.sh deployCC \
```

```
-ccn basic \
```

```
-ccp ../asset-transfer-basic/chaincode-javascript \
```

```
-ccl javascript
```

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ ./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/ch
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
```

Explanation:

This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment

Command:

```
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

```
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

Explanation:

This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger

Command:

peer chaincode invoke ... InitLedger

```
anhsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \ orderer endpoint
-o localhost:7050 \eTimeShift duration          The amount of time to shift backwards for certificate expiration
--ordererTLSHostnameOverride orderer.example.com \ent map of arguments in JSON encoding
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/tls/ca.crt
-C mychannel \050 \
-n basic \LSHostnameOverride orderer.example.com \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
-c '{"function": "InitLedger", "Args": []}'
2026-04-09 17:53:07.446 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful.
```

Explanation:

This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ ^C
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Tomoko","Size":5,"docType":"asset"}]
```

Explanation:

This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"} -c '{"Args":["ReadAsset","asset5"]}'
```

Explanation:

This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step 13: Create New Asset

Command:

peer chaincode invoke ... CreateAsset (Owner: Manav)

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function": "CreateAsset", "Args": ["asset7", "blue", "10", "Ansh", "500"]}'
2026-04-09 17:56:53.364 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"asset7","Color":"blue","Size":10,"Owner":"Ansh","AppraisedValue":500}
```

Explanation:

This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

peer chaincode invoke ... TransferAsset

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function": "TransferAsset", "Args": ["asset7", "Prajwal"]}'
2026-04-09 18:00:10.733 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Ansh"
anshsarfare16@cloudshell:~/fabric-samples/test-network$
```

Explanation:

This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step 15: Stop Network

Command:

./network.sh down

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
project name can't be empty. Use `--project-name` to set a valid name
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
8d9ff7fcda7d
56fc5fb4c185
f2319c6bca6e
2124d708dafa
e5fa71c08b55
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c
Deleted: sha256:23d8f6316d1d5a22d7fcd629ae36ca1426a24e36568682ae23125476b737b922
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c
Deleted: sha256:d991d4d6e97e8ce6d8e1fe329c0eed56f788b29216a1801fe41ea8125c8911c6
anshsarfare16@cloudshell:~/fabric-samples/test-network$
```

Explanation:

This step stops all running Docker containers and shuts down the Hyperledger Fabric network. It frees system resources and safely ends the experiment.

Result:

The Hyperledger Fabric network was successfully implemented. Chaincode was deployed and asset operations such as creation, querying, and transfer were performed successfully. The asset was created with owner name **Manav**, confirming correct execution.

Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.